

Some Fundamental Concepts :-

The Processing unit, which executes machine instructions and coordinates the activities of other units. This unit is often called the Instruction Set Processor (ISP), or simply the Processor.

To execute a program, the processor fetches one instruction at a time and performs the operations specified. Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered.

To execute an instruction, the processor has to perform the following three steps:

1. Fetch the contents of the memory location pointed to by the PC. The contents of this location are interpreted as an instruction to be executed. Hence, they are loaded into the IR. Symbolically, this can be written as

$$\boxed{IR \leftarrow [[PC]]}$$

2. Assuming that the memory is byte addressable, increment the contents of the PC by 4, that is,

$$PC \leftarrow [PC] + 4$$

3. Carry out the actions specified by the instruction in the IR.

The steps 1 and 2 are referred to as Fetch Phase and Step 3 is referred to as Execution Phase.

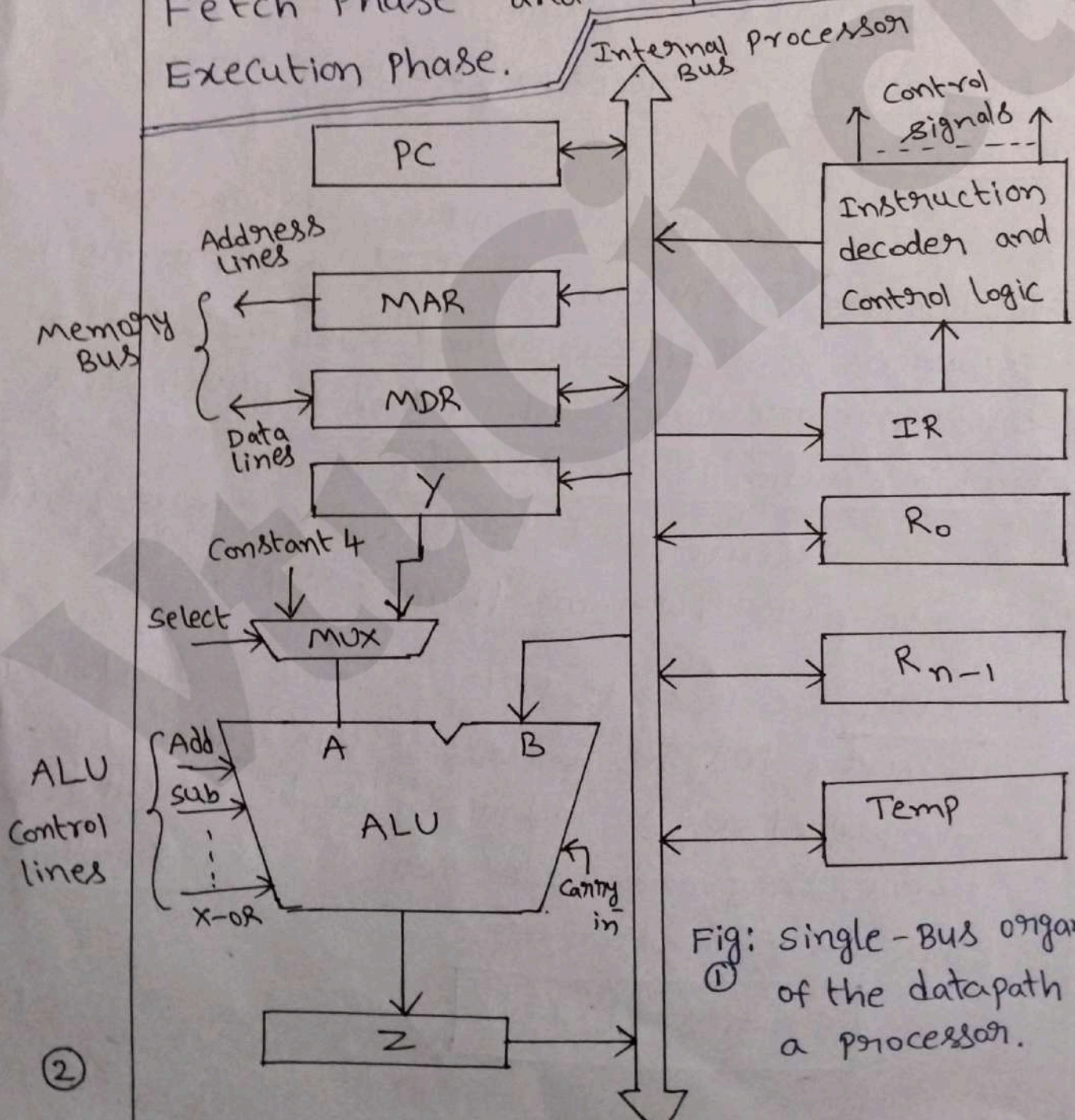


Fig: Single-Bus organization of the datapath inside a processor.

The Figure ① shows an organization in which the arithmetic and logic unit (ALU) and all the registers are interconnected via a single common-bus. This bus is internal to the processor and should not be confused with the external bus that connects the processor to the memory and I/O devices.

The data and address lines of the external memory bus are shown in Figure ① connected to the Internal Processor Bus via the Memory Data Register (MDR), and the Memory-Address Register (MAR), respectively. Register MDR has two inputs and two outputs. Data may be loaded into MDR either from the memory bus or from the internal processor bus. The data stored in MDR may be placed on either bus. The input of MAR is connected to the internal bus, and its output is connected to the external bus. The control lines of the memory bus are connected to the instruction decoder and control logic block. This unit is responsible for issuing the signals that control the operation of all the units inside the processor and for interacting with the memory bus.

The number and use of the Processor - registers R_0 through R_{n-1} vary considerably from one processor to another. Registers may be provided for general-purpose use by the Programmer.

Some may be dedicated as Specif - purpose registers, such as index registers on Stack pointers.

Three registers, XZ, and TEMP in Figure (1) have not been mentioned before. These registers are transparent to the Programmer, that is, the Programmer need not be concerned with them because they are never referenced explicitly by any instruction. They are used by the Processor for temporary storage during execution of some instructions. These registers are never used for storing data generated by one instruction for later use by another instruction.

The multiplexer (MUX) selects either the output of register Y on a constant value 4 to be provided as input A of the ALU. The constant 4 is used to increment the contents of the Program Counter. We will refer to the two possible values of the MUX control input Select as Select 4 and Select Y for selecting the constant 4 on register Y, respectively.

As instruction execution progresses, data are transferred from one register to another, often passing through the ALU to perform some arithmetic or logic operation. The instruction decoder and control logic units is responsible for implementing the actions specified by the instruction loaded in the IR register. The decoder generates the control signals needed to select the registers involved and direct the transfer of data. The registers, the ALU, and the inter connecting bus are collectively referred to as the datapath.

with few exceptions, an instruction can be executed by performing one or more of the following operations in some specified sequence:

- Transfer a word of data from one processor register to another or to the ALU.
- Perform an arithmetic or a logic operation and store the result in a processor register.
- Fetch the contents of a given memory location and load them into a processor registers.
- Store a word of data from a processor registers into a given memory location.

Register Transfers:-

Instruction execution involves a sequence of steps in which data are transferred from one register to another. For each register, two control signals are used to place the contents of that register on the bus or to load the data on the bus into the register.

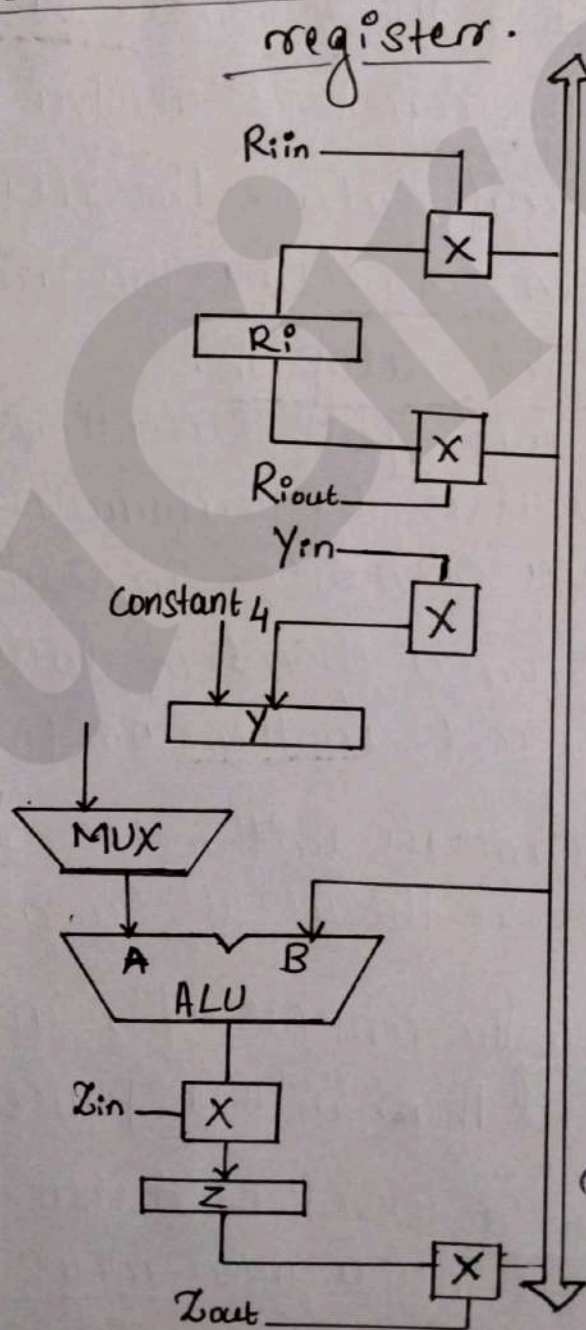


Figure (2)
Input and output
going for the
registers in
figure.

This is represented symbolically in Figure 2. The input and output of register R_i are connected to the bus via switches controlled by the signals R_{in} and R_{out} , respectively. When R_{in} is set to 1, the data on the bus are loaded into R_i . Similarly, when R_{out} is set to 1, the contents of register R_i are placed on the bus. While R_{out} is equal to 0, the bus can be used for transferring data from other registers.

Suppose that we wish to transfer the contents of register R_1 to register R_4 . This can be accomplished as follows:

- Enable the output of register R_1 by setting R_{out} to 1. This places the contents of R_1 on the processor bus.
- Enable the input of register R_4 by setting R_{in} to 1. This loads data from the processor bus into register R_4 .

All operations and data transfers within the processor take place within time periods defined by the processor clock. The control signals that govern a particular transfer are asserted at the start of the clock cycle.

When edge-triggered flip-flops are not used, two or more clock signals may be needed to guarantee proper transfer of data. This is known as multiphase clocking.

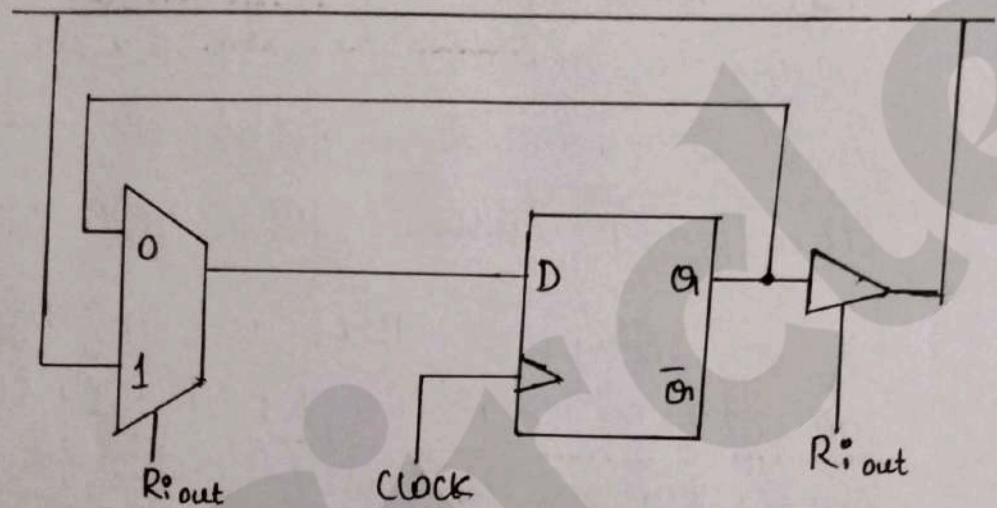


Figure ③ Input and output gating for one register bit.

An implementation for one bit of register R_i is shown in Figure ③ as an example. A two-input multiplexer is used to select the data applied to the input of an edge-triggered D Flip-flop. When the control input R_{in} is equal to 1, the multiplexer selects the data on the bus. This data will be loaded into the flip-flop at the rising edge of the clock. When R_{in} is equal to 0, the multiplexer feeds back the value currently stored in the flip-flop.

The Θ output of the flip-flop is connected to the bus via a tri-state gate. When R_{iout} is equal to 0, the gate's output is in the high-impedance (electrically disconnected) state. This corresponds to the open-circuit state of a switch. When $R_{iout} = 1$, the gate drives the bus to 0 or 1, depending on the value of Θ .

Performing An Arithmetic Or Logic Operators:-

The ALU is a combinational circuit that has no internal storage. It performs arithmetic and logic operations on the two operands applied to its A and B inputs. In Figure (1) and Figure (2). One of the operands is the output of the multiplexer MUX and the other operand is obtained directly from the bus. The result produced by the ALU is stored temporarily in register Z. Therefore, a sequence of operations to add the contents of register R_1 to those of register R_2 and store the result in register R_3 is

1. R_{1out}, Y_{in}
2. $R_{2out}, \text{Select } y, \text{Add}, Z_{in}$
3. Z_{out}, R_{3in}

Fetching A Word From Memory:-

To fetch a word of information from memory, the processor has to specify the address of the memory location where this information is stored and request a Read operation. This applies whether the information to be fetched represents an instruction in a Program or an operand specified by an instruction. The processor transfers the required address to the MAR, whose output is connected to the address lines of the memory bus to indicate that a Read operation is needed. At the same time, the processor uses the control lines of the memory bus to indicate that a Read operation is needed. When the requested data are received from the memory they are stored in register MDR, from where they can be transferred to other registers in the processor.

The connections for register MDR are illustrated in Figure (4). It has four control signals: MDR_{in} & MDR_{out} control the connection to the internal bus, and MDR_{inE} and MDR_{outE} control the connection to the external bus. The circuit in Figure is easily modified to provide the additional connections.

A three-input multiplexer can be used, with the memory bus data line connected to the third input. This input is selected when $MDR_{inE} = 1$. A second tri-state gate, controlled by MDR_{outE} can be used to connect the output of the flip-flop to the memory bus.

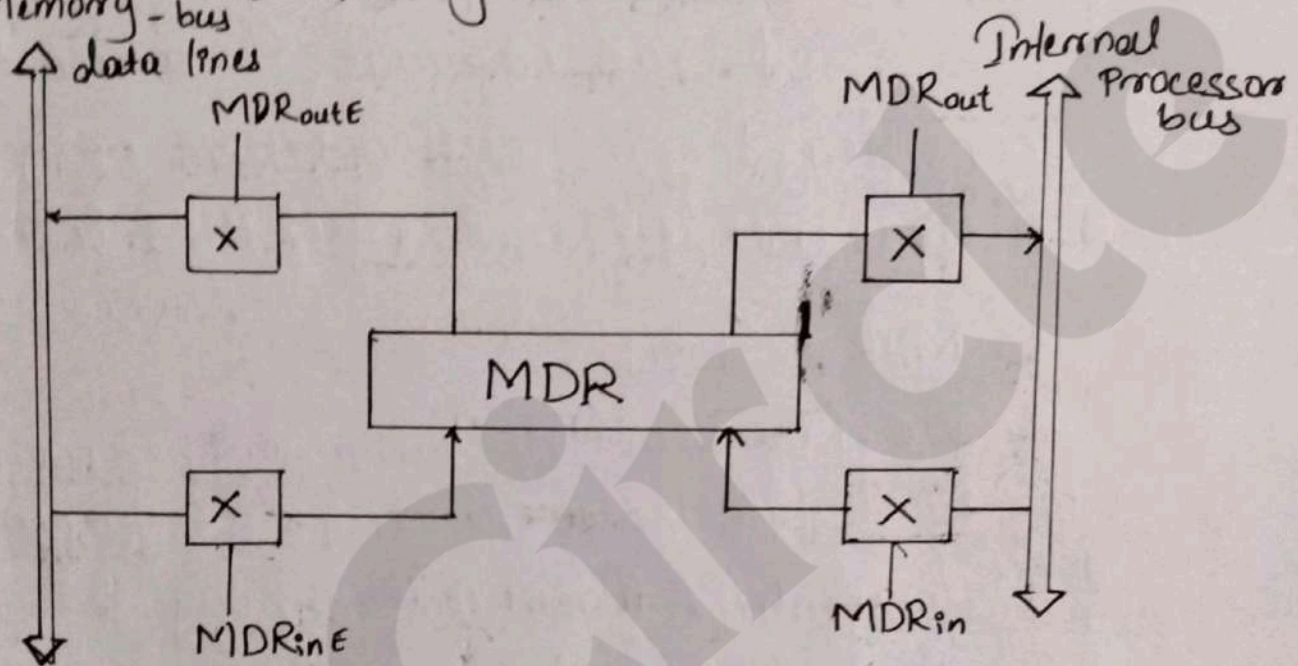


Figure (4) Connection and control signals for register MDR.

To accommodate the variability in response time, the processor waits until it receives an indication that the requested Read operation has been completed. We will assume that a control signal called Memory-Function-Completed (MFC) is used for this purpose. The addressed device sets this signal to 1 to indicate that the contents of the specified location

have been read and are available on the data lines of the memory bus. we encountered several examples of such as a signal in conjunction with the buses discussed. Such as Slave-ready.

As an example of a read operation consider the instruction MOVE (R_1), R_2 . The actions needed to execute this instruction are

1. $MAR \leftarrow [R_1]$
2. Start a Read operation on the memory bus.
3. Wait for the MFC response from the memory
4. Load MDR from the memory bus.
5. $R_2 \leftarrow [MDR]$

The memory read operation requires three steps, which can be described by the signals being activated as follows:

1. R_{1out} , MAR_{in} , Read
2. MDR_{inE} , $WMFC$
3. MDR_{out} , R_{2in}

Storing A Word In Memory :-

writing a word into a memory location follows a similar procedure. The desired address is loaded into MAR. Then the data to be written are loaded into MDR, and a write command is issued. Hence, executing the instruction Move $R_2, (R_1)$ requires the following sequence:

1. R_{2out}, MAR_{in}
2. $R_{2out}, MDR_{in}, Write$
3. $MDR_{out}, WMFC$.

As in the case of the read operation. The write control signal cause the memory bus interface hardware to issue a write command on the memory bus. The processor remains in step 3 until the memory operation is completed and an MFC response is received.

Execution of A Complete Instruction:-

The Sequence of elementary operations required to execute one instruction. Consider the instruction

Add (R₃), R₁

which adds the contents of a memory location pointed to by R₃ to register R₁. Executing this instruction requires the following actions:

1. Fetch the instruction.
2. Fetch the first operand (the contents of the memory location pointed to by R₃).
3. Perform the addition.
4. Load the result into R₁.

Figure gives the sequence of control steps required to perform these operations for the single-bus architecture of Figure (8). Instruction execution proceeds as follows. In step 1, the instruction fetch operation is initiated by loading the contents of the PC into the MAR and sending a Read requests to the memory. The Select signal is set to Select 4, which causes the multiplexer MUX to select the constant 4. This value is added to the operand at input B,

which is the contents of the PC, and the result is stored in register Z. The updated value is moved from register Z back into the PC during step 2, while waiting for the memory to respond. In step 3, the word fetched from the memory is loaded into the IR.

Steps 1 through 3 constitute the instruction fetch phase, which is the same for all instructions. The instruction decoding circuit interprets the contents of the IR at the beginning of steps 4. This enables the control circuitry to activate the control signals for steps 4 through 7, which constitute the execution phase. The contents of register R3 are transferred to the MAR in step 4, and a memory read operation is initiated. Then

Step	Action
1	PC _{out} , MAR _{in} , Read, Select 4, Add1, Z _{in}
2	Z _{out} , PC _{in} , Y _{in} , WMFC
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMFC
6	MDR _{out} , Select y, Add1, Z _{in}
7	Z _{out} , R1 _{in} , End

Fig (5)

the contents of R_1 are transferred to register Y in Step 5, to prepare for the addition operation. When the Read operation is completed, the memory operand is available in register MDR, and the addition operation is performed in Step 6. The contents of MDR are gated to the bus, and thus also to the B input of the ALU, and register Y is selected as the second input to the ALU by choosing Select Y . The sum is stored in register Z , then transferred to R_1 in the Step 7. The End signal causes a new instruction fetch cycle to begin by returning to Step 1.

This discussion accounts for all control signals in Figure (5) except Yin in Step 2. There is no need to copy the updated contents of PC into register Y when executing the Add instruction. But, in Branch instructions the updated value of the PC is needed to compute the Branch target address. To speed up the execution of Branch instructions, this value is copied into register Y in Step 2. Since Step 2 is part of the fetch phase, the same action will be performed for all instructions. This does not cause any harm because register Y is not used for any other purpose at the time.

Pipelining: Basic Concepts

The speed of execution of programs is influenced by many factors. One way to improve performance is to use faster circuit technology to build the processors and the main memory. Another possibility is to arrange the hardware so that more than one operation can be performed at the same time. In this way, the number of operations performed per second is increased even though the elapsed time needed to perform any one operation is not changed.

Pipelining is a particularly effective way of organizing concurrent activity in a computer system. The basic idea is very simple. It is frequently encountered in manufacturing plants, where pipelining is commonly known as an assembly-line operation.

Readers are undoubtedly familiar with the assembly line used in car manufacturing. The first station in an assembly line may prepare the chassis of a car, the next station adds the body, the next one installs the engine, and so on. While one group of workers is installing the engine on one car, another group is fitting a car body on the chassis of another car, and yet another group is preparing a new chassis for a third car. It may take days to complete work on a given car, but it is possible to have a new car rolling off the end of the assembly line every few minutes.

Consider how the idea of pipelining can be used in a computer. The processor executes a program by fetching and executing instructions, one after the other, let F_i and E_i refer to the fetch and execute steps for instruction I_i . Execution of a program consists of a sequence of fetch and execute steps, as shown in Figure (1)

Now consider a computer that has two separate hardware units, one for fetching instructions and another for executing them, as shown in Figure (1). The instruction fetched by the fetch unit is deposited in an intermediate storage buffer, B1. This buffer is needed to enable the execution unit to execute the instruction while the fetch unit is fetching the next instruction. The results of execution are deposited in the destination location specified by the instruction. For the purposes of this discussion, we assume that both the source and the destination of the data operated on by the instructions are inside the block labeled "Execution unit."

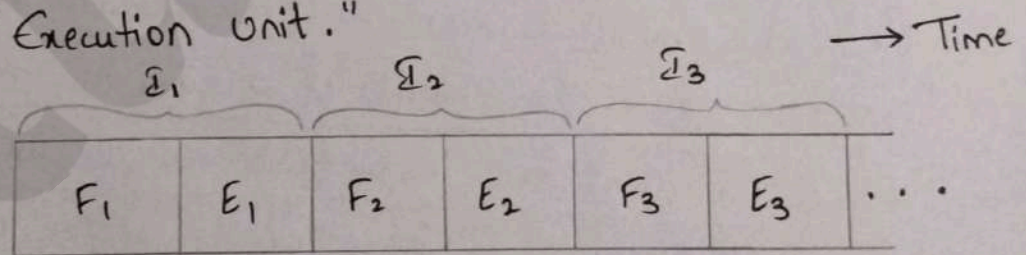


Fig (1) Sequential Execution

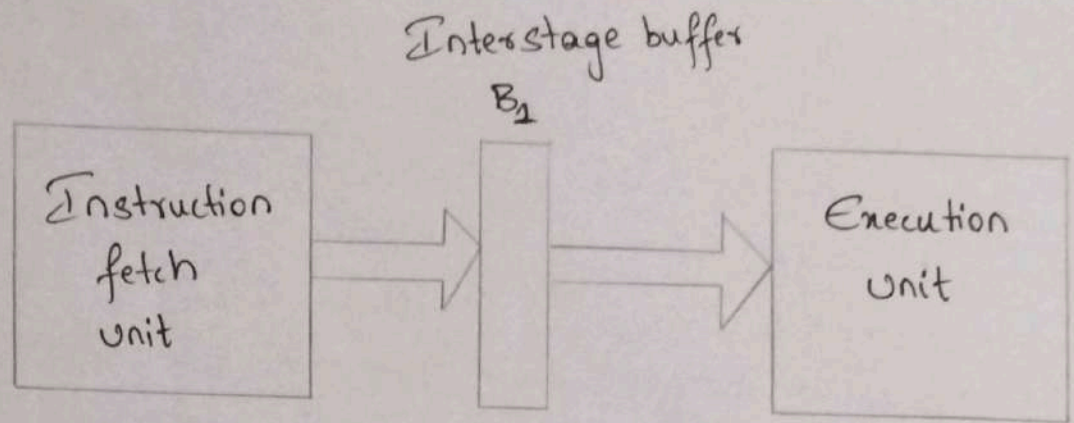


Fig (1): Hardware Organization

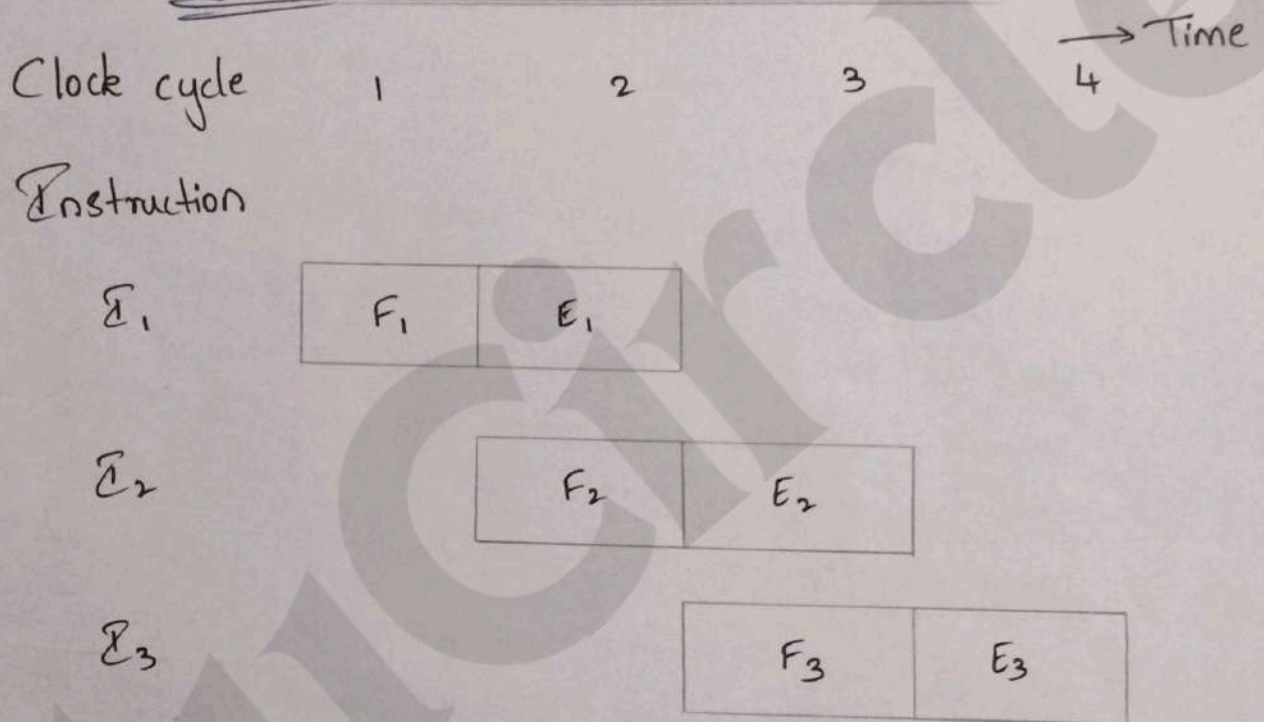


Fig. (b) Pipelined Execution

Fig. (2) Basic idea of Instruction pipelining

The computer is controlled by a clock whose period is such that the fetch and execute steps of any instruction can each be completed in one clock cycle. Operation of the computer proceeds as in Figure (2).

(19)

(18)

In the first clock cycle, the fetch unit fetches an instruction I_1 (step F_1) and stores it in buffer B_1 at the end of the clock cycle. In the second clock cycle, the instruction fetch unit proceeds with the fetch operation for instruction I_2 (step F_2). Meanwhile, the execution unit performs the operation specified by instruction I_1 , which is available to it in buffer B_1 (step E_1). By the end of the second clock cycle, the execution of instruction I_1 is completed and instruction I_2 is available. Instruction I_2 is stored in B_1 , replacing I_1 , which is no longer needed. Step E_2 is performed by the execution unit during the third clock cycle, while instruction I_3 is being fetched by the fetch unit. In this manner, both the fetch and execute units are kept busy all the time. If the pattern in figure 2(a) can be sustained for a long time, the completion rate of instruction execution will be twice that achievable by the sequential operation depicted in Figure

In summary, the fetch and execute units in Figure 2(b) constitute a two-stage pipeline in which each stage performs one step in processing an instruction. An interstage storage buffer, B_1 , is needed to hold the information being passed from one stage to the next. New information is loaded into this buffer at the end of each clock cycle.

The processing of an instruction need not be divided into only two steps. For example, a pipelined processor may process each instruction in four steps, as follows:

- 1) F Fetch: read the instruction from the memory
- 2) D Decode: decode the instruction and fetch the source operand(s).
- 3) E Execute: perform the operation specified by the instruction
- 4) W Write: store the result in the destination location.

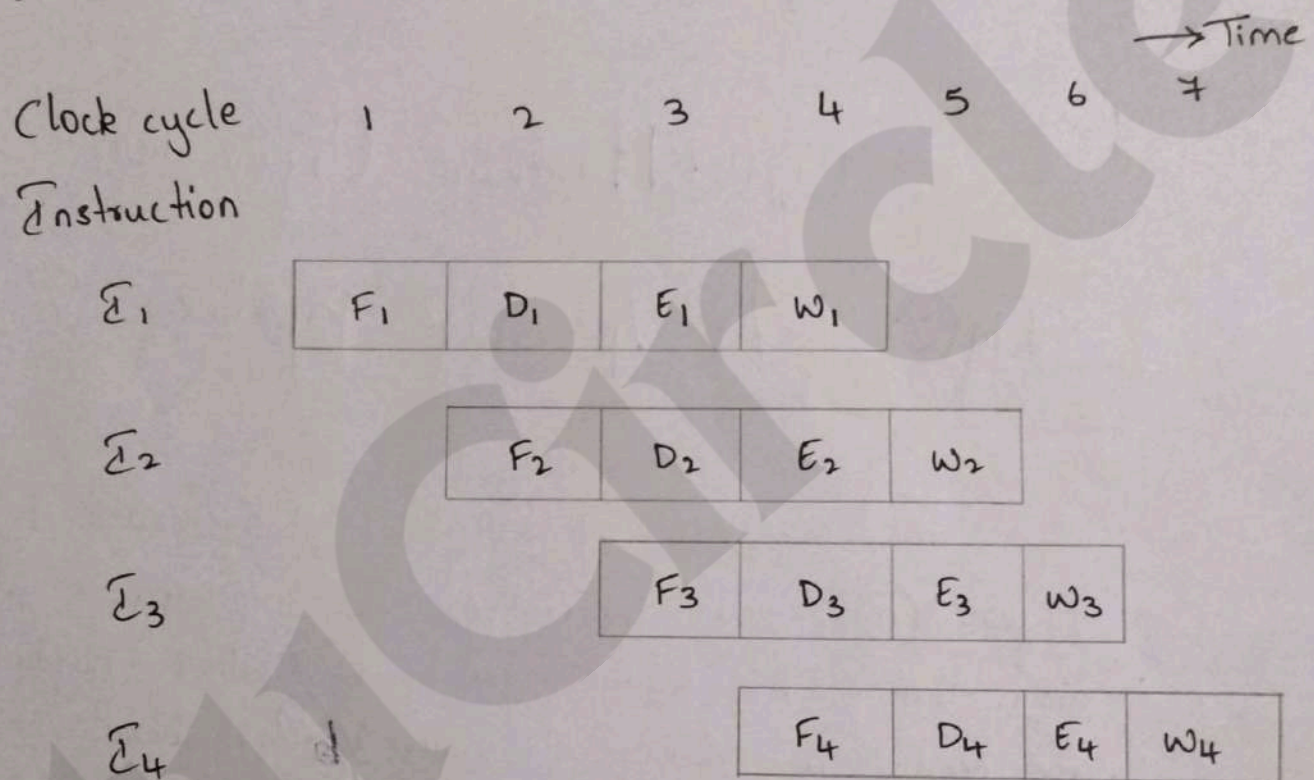


Fig 3 (a) Instruction execution divided into four steps

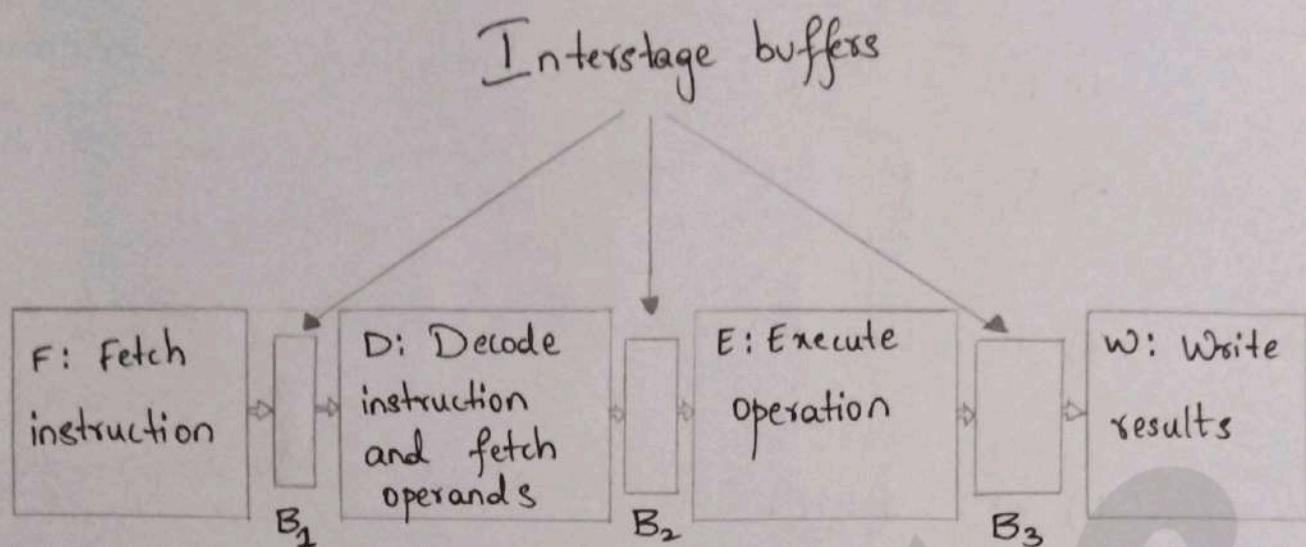


Fig ⑥ Hardware Organization

Fig ③ : A 4-Stage Pipeline

The sequence of events for this case is shown in Figure 3(a). Four instructions are in progress at any given time. This means that four distinct hardware units are needed, as shown in Figure 3(b). These units must be capable of performing their tasks simultaneously and without interfering with one another. Information is passed from one unit to the next through a storage buffer. As an instruction progresses through the pipeline, all the information needed by the stages downstream must be passed along. For example, during clock cycle 4, the information in the buffers is as follows:

- 1) Buffer B₁ holds instruction I₃, which was fetched in cycle 3 and is being decoded by the instruction-decoding unit.

2) Buffer B_2 holds both the source operands for instruction I_2 and the specification of the operation to be performed. This is the information produced by the decoding hardware in cycle 3. The buffer also holds the information needed for the write step of instruction I_2 (step w_2). Even though it is not needed by stage E, this information must be passed on to stage W in the following clock cycle to enable that stage to perform the required Write operation.

3) Buffer B_3 holds the results produced by the execution unit and the destination information for instruction I_1 .

ROLE OF CACHE MEMORY

Each stage in a pipeline is expected to complete its operation in one clock cycle. Hence, the clock period should be sufficiently long to complete the task being performed in any stage. If different units require different amounts of time, the clock period must allow the longest task to be completed. A unit that completes its task early is idle for the remainder of the clock period. Hence, pipelining is most effective in improving performance if the tasks being performed in different stages require about the same amount of time.

This consideration is particularly important for the instruction fetch step, which is assigned one clock period in Figure 3 (a). The clock cycle has to be equal to or greater than the time needed to complete a fetch operation. However,

the access time of the main memory may be as much as ten times greater than the time needed to perform basic pipeline stage operations inside the processor, such as adding two numbers. Thus, if each instruction fetch required access to the main memory, pipelining would be of little value.

The use of cache memories solves the memory access problem. In particular, when a cache is included on the same chip as the processor, access time to the cache is usually the same as the time needed to perform other basic operations inside the processor. This makes it possible to divide instruction fetching and processing into steps that are more or less equal in duration. Each of these steps is performed by a different pipeline stage, and the clock period is chosen to correspond to the longest one.

PIPELINE PERFORMANCE

The pipeline processor in Figure 3(a) completes the processing of one instruction in each clock cycle, which means that the rate of instruction processing is four times that of sequential operation. The potential increase in performance resulting from pipelining is proportional to the number of pipeline stages. However, this increase would be achieved only if pipelined operation as depicted in Figure 3(a) could be sustained without interruption throughout program execution. Unfortunately, this is not the case.

For a variety of reasons, one of the pipeline stages may not be able to complete its processing task for a given instruction in the time allotted. For example, stage E in the four-stage pipeline of Figure 3(b) is responsible for arithmetic and logic operations, and one clock cycle is assigned for this task. Although this may be sufficient for most operations, some operations, such as divide, may require more time to complete. Figure 4(p) shows an example in which the operation specified in instruction I_2 requires three cycles to complete, from cycle 4 through cycle 6. Thus, in cycles 5 and 6, the write stage must be told to do nothing, because it has no data to work with. Meanwhile, the information in buffer B_2 must remain intact until the Execute stage has completed its operation. This means that stage 2 and, in turn, stage 1 are blocked from accepting new instructions because the information in B_1 cannot be overwritten. Thus, steps D_4 and F_5 must be postponed as shown.

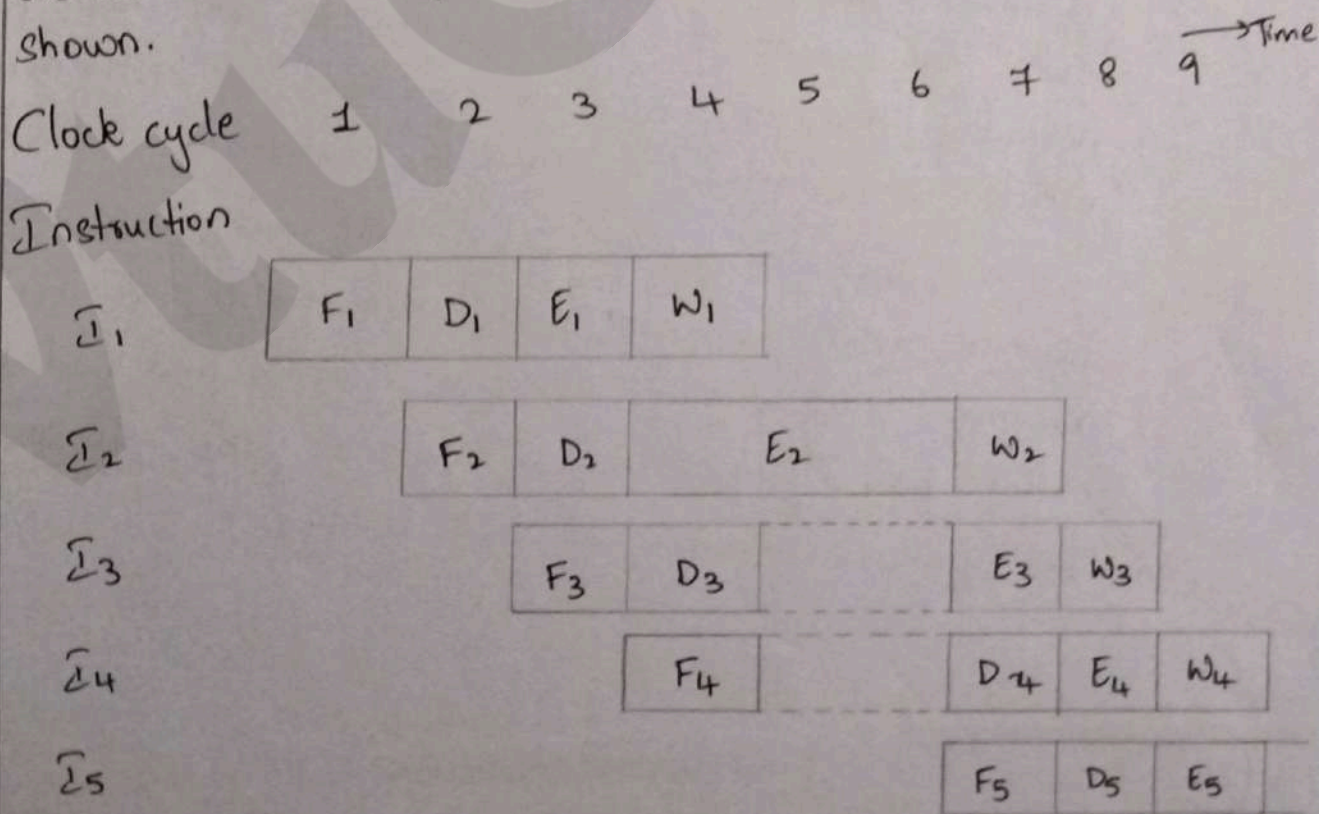


Fig 4(p) Effect of an execution operation taking more than one clock cycle.

Pipelined operation in Figure is said to have been stalled for two clock cycles. Normal pipelined operation resumes in cycle 4. Any condition that causes the pipeline to stall is called a hazard. We have just seen an example of a data hazard. A data hazard is any condition in which either the source or the destination operands of an instruction are not available at the time expected in the pipeline. As a result some operation has to be delayed, and the pipeline stalls.

The pipeline may also be stalled because of a delay in the availability of an instruction. For example, this may be a result of a miss in the cache, requiring the instruction to be fetched from the main memory. Such hazards are often called control hazards or instruction hazards. The effect of a cache miss on pipelined operation is illustrated in Figure (3). Instruction I_1 is fetched from the cache in cycle 1, and its execution proceeds normally. However, the fetch operation for instruction I_2 , which is started in cycle 2, results in a cache miss.

The instruction fetch unit must now suspend any further fetch requests and wait for I_2 to arrive. We assume that instruction I_2 is received and loaded into buffer B_1 at the end of cycle 5. The pipeline resumes its normal operation at that point.

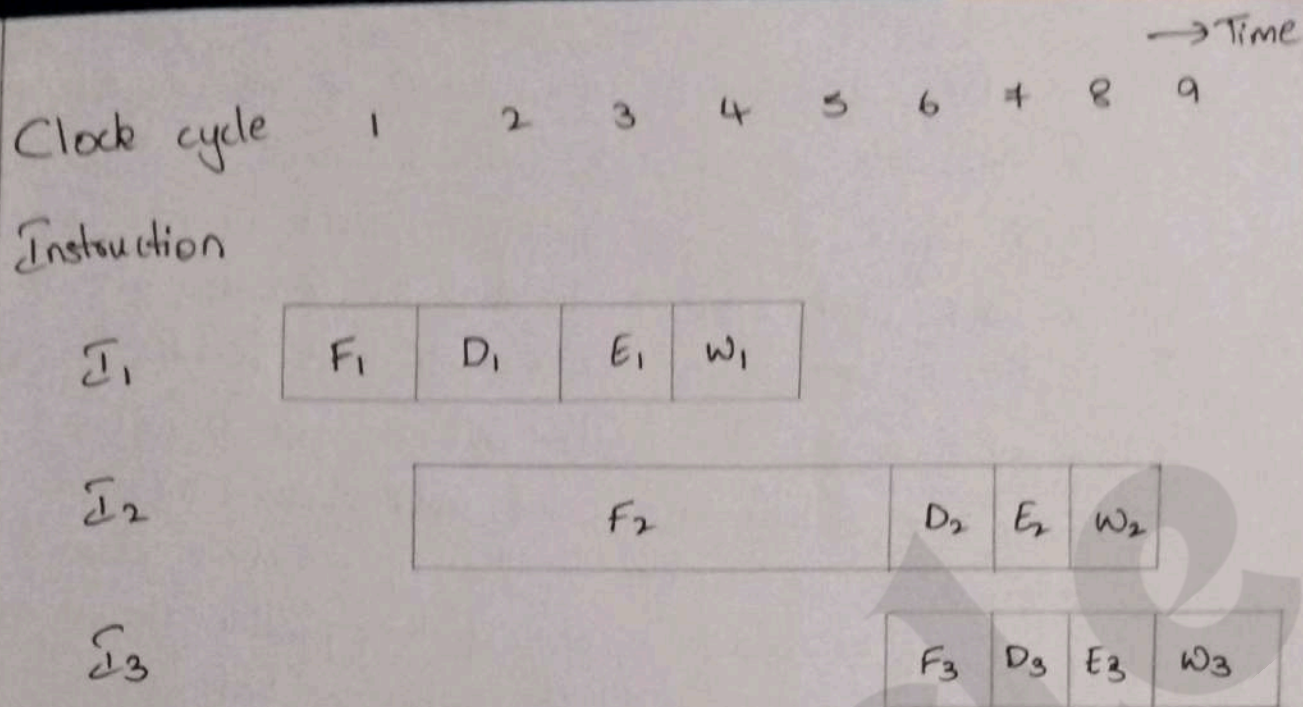


Fig. 5(a) Instruction execution step in successive clock cycles

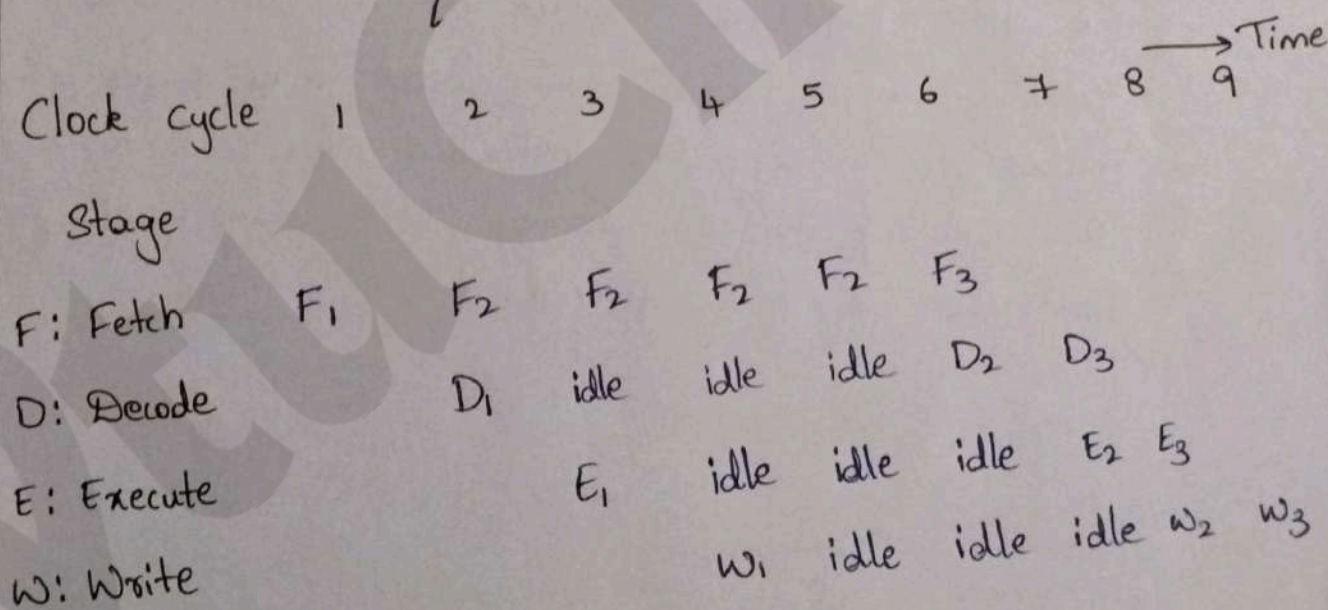


Fig. 5(b) Function performed by each processor stage in successive clock cycles

Fig. 5(c) Pipeline stall caused by a cache miss in F₂

An alternative representation of the operation of a pipeline in the case of a cache miss is shown in Figure. This figure gives the function performed by each pipeline stage in each clock cycle. Note that the Decode unit is idle in cycles 3 through 5, the Execute unit is idle in cycles 4 through 6, and the Write unit is idle in cycles 5 through 7. Such idle periods are called ~~units~~ stalls. They are also often referred to as bubbles in the pipeline. Once created as a result of a delay in one of the pipeline stages, a bubble moves downstream until it reaches the last unit.

An third type of hazard that may be encountered in pipelined operation is known as a structural hazard. This is the situation when two instructions require the use of a given hardware resource at the same time. The most common case in which this hazard may ~~arise~~ arise is in access to memory. One instruction may need to access memory as part of the Execute or Write stage while another instruction is being fetched.